

Exercícios : qual método é chamado?

Eduardo Barella 24088000-2

Matheus Mascarello 24120740-2

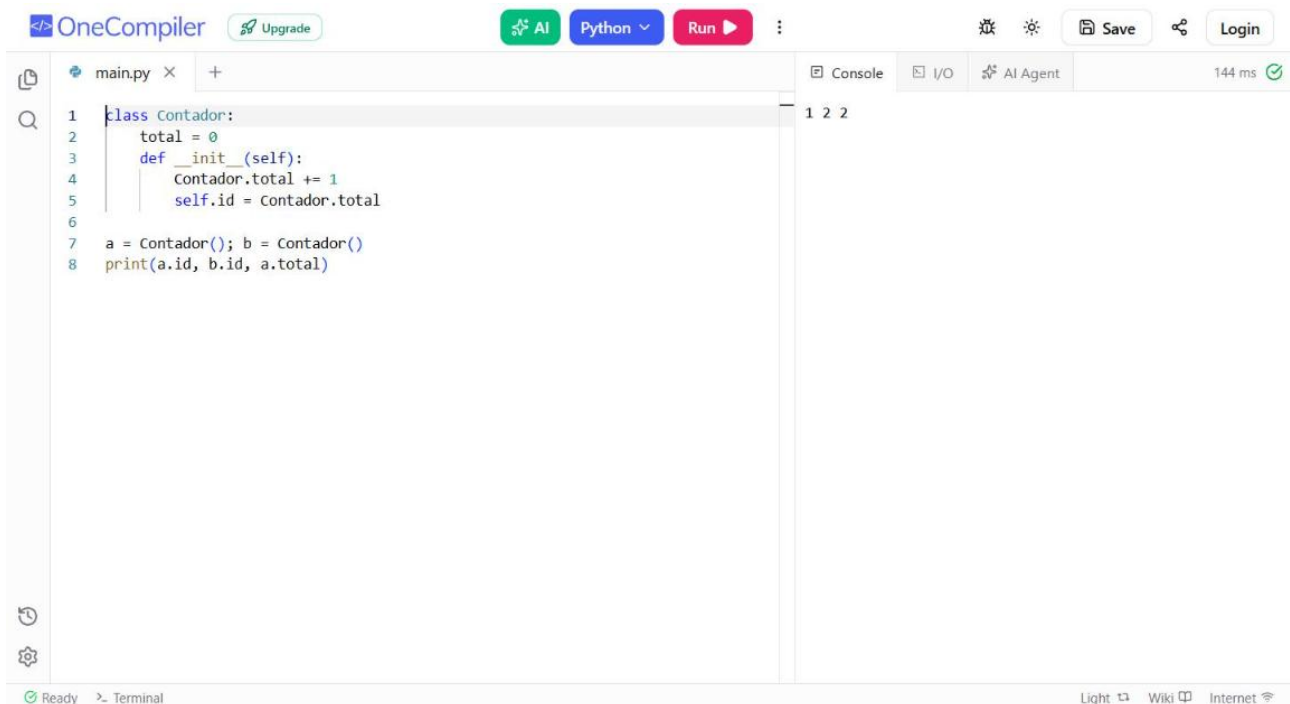
Exercício 1/3 · Python

```
class Contador:
    total = 0
    def __init__(self):
        Contador.total += 1
        self.id = Contador.total
```

```
a = Contador(); b = Contador()
print(a.id, b.id, a.total)
```

(a) Saída: 1 2 2

(b) **Compilação ou execução? Execução.** Em Python tudo é resolvido em tempo de execução. `total` é um atributo da **classe** (compartilhado por todos os objetos), e `id` é um atributo de cada **instância**. Cada construtor incrementa `Contador.total`: `a` recebe `id` 1 e `b` recebe `id` 2. Como `a` não tem atributo próprio `total`, a busca em `a.total` sobe para a classe e encontra 2 (variável de classe × variável de instância, Sebesta §12.3).



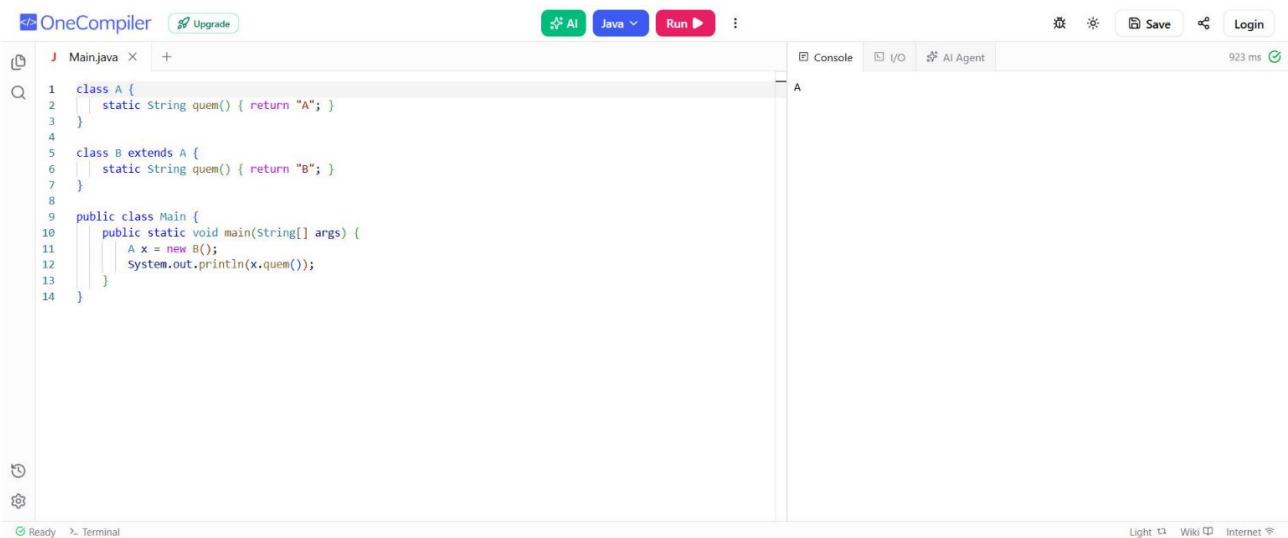
Exercício 2/3 · Java

```
class A { static String quem() { return "A"; } }  
class B extends A { static String quem() { return "B"; } }
```

```
A x = new B();  
System.out.println(x.quem());
```

(a) Saída: A

(b) **Compilação ou execução? Compilação.** Métodos static pertencem à classe, não ao objeto, e não participam da vinculação dinâmica: em Java, só static, private e final ficam de fora do despacho dinâmico (Sebesta §12.3.7, p. 497). O quem() de B apenas **esconde** o de A (não sobrescreve), e a chamada x.quem() é decidida pelo tipo **declarado** da variável (A), não pelo tipo do objeto (B). Por isso o recomendado é chamar A.quem(), e não pelo objeto.



Exercício 3/3 · Go

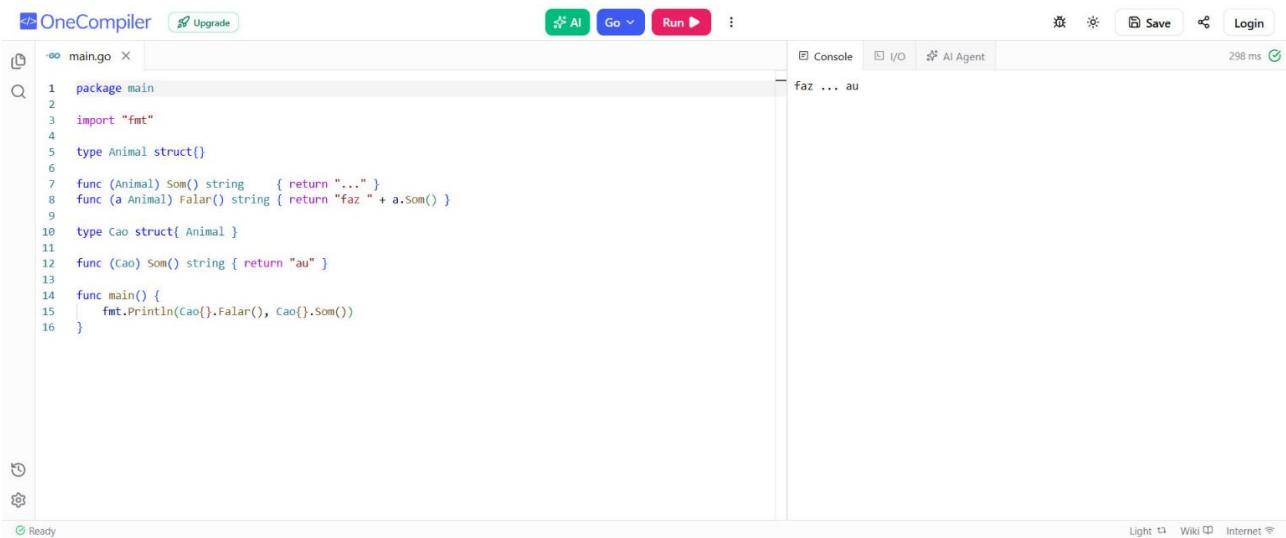
```
type Animal struct{}
func (Animal) Som() string { return "..."}
func (a Animal) Falar() string { return "faz " + a.Som() }
```

```
type Cao struct{ Animal }
func (Cao) Som() string { return "au" }
```

```
fmt.Println(Cao{}.Falar(), Cao{}.Som())
```

(a) Saída: faz ... au

(b) **Compilação ou execução? Compilação.** Go não tem herança: Cao só **contém** um Animal (composição por embedding). Cao{}.Falar() é promovido para Cao{}.Animal.Falar(), e dentro de Falar o receptor a é do tipo Animal, então a.Som() chama Animal.Som(...), decidido na compilação. Já Cao{}.Som() chama direto o método de Cao ("au"). Em Go, a vinculação dinâmica só acontece através de **interfaces**.



```
OneCompiler Upgrade
AI Go Run
main.go x Console I/O AI Agent 298 ms
1 package main
2
3 import "fmt"
4
5 type Animal struct{}
6
7 func (Animal) Som() string { return "..."}
8 func (a Animal) Falar() string { return "faz " + a.Som() }
9
10 type Cao struct{ Animal }
11
12 func (Cao) Som() string { return "au" }
13
14 func main() {
15     fmt.Println(Cao{}.Falar(), Cao{}.Som())
16 }
```

faz ... au

Ready Light Wiki Internet